Final Client Round — Banking Domain Questionnaire (Part 1)

Candidate: Pushparaj Naik — AWS Cloud Architect (22+ years)
Context: Final round with banking-sector client
Focus: AWS Architecture, Security & Compliance, Terraform IaC

Section 1: Banking-Specific AWS Architecture (10 Questions)

Q1. How would you design a multi-account AWS landing zone for a bank with separate regulatory environments (production banking, UAT, DR, sandbox)?

Answer:

I'd implement an **AWS Control Tower–based multi-account strategy** aligned with the AWS Well-Architected Financial Services Lens:

```
Root OU
├── Security OU
│   ├── Log Archive Account (centralized CloudTrail, Config, VPC Flow Logs)
│   └── Security Tooling Account (GuardDuty delegated admin, Security Hub)
├── Infrastructure OU
│   ├── Shared Services Account (Transit Gateway, DNS, CI/CD tooling)
│   └── Network Account (centralized egress, inspection VPCs)
├── Workloads OU
│   ├── Production Account (PCI-DSS scope – hardened)
│   ├── UAT Account (mirrors prod config, synthetic data only)
│   └── Development Account (relaxed SCPs, cost controls)
├── DR OU
│   └── DR Account (cross-region replication, pilot light/warm standby)
└── Sandbox OU
    └── Sandbox Account (auto-nuke policies, budget caps)
```

Key Banking Controls:

- **SCPs at OU level:** Block non-approved regions (data residency), deny `iam:CreateUser` with console access, enforce encryption
- **AWS Config Rules:** Mandatory across all accounts — `s3-bucket-server-side-encryption-enabled`, `rds-storage-encrypted`, `ec2-imdsv2-required`
- **Centralized logging:** All CloudTrail logs to Log Archive (immutable S3 with Object Lock for audit)
- **Network isolation:** Transit Gateway with route table segmentation — production cannot reach sandbox

In my Rio Tinto project, I implemented a similar pattern with remote state isolation per environment and cross-account IAM roles via OIDC, removing long-lived credentials

entirely.

Q2. A bank needs to process real-time transaction data with sub-second latency and store it for 7 years for regulatory compliance. How would you architect this on AWS?

Answer:

I'd design a **dual-path architecture** — hot path for real-time and cold path for compliance:

Hot Path (sub-second):

```
Transactions → API Gateway → Kinesis Data Streams → Lambda
(validation/enrichment)
                                                    → DynamoDB (real-time
ledger)
                                                    → ElastiCache Redis
(session/fraud cache)
```

Cold Path (compliance archive):

```
Kinesis Data Streams → Kinesis Firehose → S3 (Parquet, KMS-encrypted)
                                        → S3 Lifecycle (Standard → IA 90d →
Glacier 1yr → Deep Archive)
                                        → Retention: 7 years with Object
Lock (WORM compliance)
```

Analytics Layer:

```
S3 Data Lake → Glue Catalog → Athena (ad-hoc audit queries)
             → Glue ETL Jobs → Redshift (regulatory reporting)
```

Key Design Decisions:

- **Kinesis over SQS** for ordering guarantees (partition key = account_id ensures per-account ordering)

- **DynamoDB with DAX** for single-digit ms reads on account balances
- **S3 Object Lock in Compliance mode** — even root cannot delete (SEC 17a-4 / FINRA compliance)
- **KMS CMK with key rotation** — separate keys per data classification (PII, transaction, metadata)

This mirrors patterns I used at ITC Infotech with S3 data stores and Glue jobs for the Databricks-ready data platform, but with banking-specific compliance controls layered on.

Q3. How do you handle data residency requirements for a bank operating in India under RBI's data localization mandate?

Answer:

RBI's 2018 circular mandates that all payment system data must be stored **only in India**. My approach:

1. **Region Lock via SCPs:**

```
{
  "Effect": "Deny",
  "Action": "*",
  "Resource": "*",
  "Condition": {
    "StringNotEquals": {
      "aws:RequestedRegion": ["ap-south-1", "ap-south-2"]
    }
  }
}
```

2. **S3 Bucket Policies:** Explicit deny on cross-region replication for PCI-scoped buckets
3. **RDS:** Multi-AZ within ap-south-1 only; cross-region read replicas only for non-PII/non-payment data
4. **DR Strategy:** Warm standby in ap-south-2 (Hyderabad) — stays within India
5. **CloudFront:** Use regional edge caches only; disable global edge locations for sensitive APIs

6. **Audit:** AWS Config rule to detect any resource created outside approved regions + auto-remediation via SSM

Data Classification:

Classification	Storage	Replication	Encryption
Payment Data (RBI scope)	ap-south-1 only	ap-south-2 DR only	KMS CMK, mandatory
Customer PII	ap-south-1 only	ap-south-2 DR only	KMS CMK + tokenization
Analytics/Aggregated	ap-south-1 primary	Can replicate globally	KMS CMK
Public content	Any region	CloudFront global	Optional

Q4. Walk me through how you'd design a PCI-DSS compliant architecture on AWS for card payment processing.

Answer:

PCI-DSS has 12 requirements across 6 domains. My AWS mapping:

Network Segmentation (Req 1-2):

- Dedicated VPC for Cardholder Data Environment (CDE)
- Private subnets only — no public subnets in CDE VPC
- Security groups: explicit allow-list, deny all by default
- NACLs as secondary defense layer
- AWS Network Firewall for stateful inspection between tiers
- VPC Flow Logs to S3 (retained 1 year minimum)

Data Protection (Req 3-4):

- **At rest:** KMS CMK encryption for all storage (S3, RDS, EBS, DynamoDB)
- **In transit:** TLS 1.2+ enforced via ALB policies; certificate pinning for internal services

- **PAN tokenization:** Use AWS Payment Cryptography or custom tokenization Lambda
- **Key management:** Separate CMKs per data domain, automatic rotation every 365 days
- **S3 bucket policy:** Deny `s3:PutObject` without `aws:kms` encryption header

Access Control (Req 7-8):

- IAM Identity Center (SSO) with MFA enforced
- RBAC via IAM policies — no inline policies, managed policies only
- Break-glass procedure with time-limited `sts:AssumeRole` + CloudTrail alerting
- No long-lived access keys — OIDC federation for CI/CD (exactly as I implemented at Rio Tinto)

Monitoring (Req 10-11):

- CloudTrail (all regions, all accounts) → S3 with Object Lock
- GuardDuty + Security Hub with PCI-DSS standard enabled
- CloudWatch alarms on unauthorized API calls, root login, failed auth attempts
- Config Rules for continuous compliance validation
- Quarterly penetration testing using AWS-approved methodology

From my experience: At ITC Infotech, I embedded these security controls into Terraform modules as reusable patterns — KMS encryption, least-privilege IAM, security groups — so every new service is PCI-compliant by default, not as an afterthought.

Q5. How would you design high availability for a core banking application with RPO=0 and RTO<15 minutes?

Answer:

RPO=0 means zero data loss — requires synchronous replication. **RTO<15min** means near-instant failover.

Architecture:

Layer	Primary (ap-south-1)	DR (ap-south-2)	Strategy
Compute	EKS (multi-AZ)	EKS (warm standby, scaled down)	Route 53 health check failover
Database	RDS Multi-AZ (synchronous)	RDS Cross-Region Read Replica	Promote replica (RPO ≈ seconds)
Cache	ElastiCache Global Datastore	Auto-failover to secondary	Built-in replication
Storage	S3	S3 Cross-Region Replication	Eventual consistency (minutes)
DNS	Route 53	Route 53 failover routing	Health check → auto switch
Secrets	Secrets Manager	Replicated to DR region	Multi-region secret

For true RPO=0 on the database:

- Use **Amazon Aurora Global Database** — replication lag typically <1 second
- Or **DynamoDB Global Tables** for NoSQL workloads — multi-region active-active with last-writer-wins
- For absolute RPO=0: Aurora with write forwarding + application-level dual-write pattern

Automated Failover Runbook:

1. CloudWatch alarm detects primary unhealthy
2. Lambda triggers Step Functions failover workflow
3. Promote Aurora read replica to writer (< 1 min)
4. Scale up EKS nodes in DR region (pre-warmed node group)
5. Route 53 health check automatically routes traffic to DR
6. SNS notification to on-call team
7. Total RTO: ~10-12 minutes

Q6. How do you handle secrets management for a banking application across multiple

environments?

Answer:

I implement a **hierarchical secrets strategy** using AWS Secrets Manager:

Naming Convention:

```
/{environment}/{service}/{secret-type}
/prod/payment-service/db-credentials
/prod/payment-service/api-keys
/uat/payment-service/db-credentials
```

Access Control:

- Secrets Manager resource policy restricts access by environment — prod secrets only accessible by prod IAM roles
- Application IAM roles use `secretsmanager:GetSecretValue` with resource ARN scoped to their environment
- No cross-environment secret access

Rotation:

- Automatic rotation via Lambda every 30 days for database credentials
- API keys rotated every 90 days
- Rotation Lambda runs in same VPC as RDS (private subnet)

In Terraform (my pattern from Rio Tinto):

```
resource "aws_secretsmanager_secret" "db_creds" {
  name       = "/${var.environment}/${var.service}/db-credentials"
  kms_key_id = aws_kms_key.secrets.arn

  # Prevent accidental deletion
  recovery_window_in_days = 30
}

resource "aws_secretsmanager_secret_rotation" "db_creds" {
  secret_id           = aws_secretsmanager_secret.db_creds.id
  rotation_lambda_arn = aws_lambda_function.secret_rotation.arn
  rotation_rules {
    automatically_after_days = 30
  }
}
```

Key Banking Controls:

- KMS CMK encryption (not default key)
- CloudTrail logging of every `GetSecretValue` call
- No secrets in environment variables, SSM parameters for non-sensitive config
- CI/CD pipelines use OIDC — no stored AWS credentials (implemented at Rio Tinto with GitHub Actions)

Q7. A banking client wants to migrate from on-premises Oracle to AWS. What's your migration strategy?

Answer:

I'd follow a **phased approach** using the AWS Migration Acceleration Program (MAP) framework:

Phase 1: Assess (2-4 weeks)

- AWS Schema Conversion Tool (SCT) to analyze Oracle → target compatibility
- Identify: PL/SQL stored procedures, Oracle-specific features (RAC, Data Guard, Materialized Views)
- Decision matrix:

Factor	Aurora PostgreSQL	RDS Oracle	Decision Driver
License cost	✅ No Oracle license	❌ BYOL or license-included	Cost savings of 60-80%
Oracle features used	Needs code refactoring	✅ Compatible	Complexity of PL/SQL
Performance	✅ 5x throughput vs standard PG	✅ Native performance	Benchmark results
Banking compliance	✅ PCI/SOX compliant	✅ PCI/SOX compliant	Both qualify

Phase 2: Schema Conversion (4-6 weeks)

- SCT converts 80-90% automatically
- Manual conversion for: Oracle-specific packages, CONNECT BY queries, advanced PL/SQL
- Parallel testing with production-equivalent data volumes

Phase 3: Data Migration (2-4 weeks)

- AWS DMS with CDC (Change Data Capture) for zero-downtime migration
- Full load → CDC replication → validation → cutover
- Validation: row counts, checksum comparison, application-level functional tests

Phase 4: Cutover (1 weekend)

- Stop application writes → wait for DMS CDC lag = 0 → switch DNS → validate → go live
- Rollback plan: keep Oracle running for 2 weeks post-cutover

In my Advantest project, I designed multi-AZ RDS with DMS-based migration for database workloads — the same pattern applies here but with banking-specific data validation requirements.

Q8. How would you implement a zero-trust network architecture on AWS for a bank?

Answer:

Zero-trust means **"never trust, always verify"** — every request is authenticated and authorized regardless of network location.

Implementation Layers:

1. **Identity-Centric Access:**
    - AWS IAM Identity Center with MFA for all human access
    - IRSA (IAM Roles for Service Accounts) on EKS — pods get scoped IAM roles, not node roles
    - Short-lived credentials everywhere (STS, OIDC federation)
2. **Network Micro-Segmentation:**
    - Security groups as the primary enforcement — one SG per service/tier
    - No broad CIDR rules — reference security groups instead (`source_security_group_id`)
    - AWS PrivateLink for all AWS service access (no internet gateway in production VPC)
    - VPC endpoints for S3, DynamoDB, KMS, Secrets Manager, ECR
3. **Service Mesh (EKS):**
    - Istio or AWS App Mesh for mTLS between all microservices
    - Network policies (Calico) to deny all traffic by default, explicit allow-list
    - Each pod identity verified via SPIFFE/SPIRE or IRSA
4. **Data Plane:**
    - All data encrypted in transit (TLS 1.2+ enforced)
    - All data encrypted at rest (KMS CMK)
    - S3 bucket policies deny unencrypted uploads
    - RDS SSL enforcement: `rds.force_ssl = 1`
5. **Continuous Verification:**
    - GuardDuty for threat detection
    - AWS Config for continuous compliance
    - CloudTrail + Athena for security analytics
    - Automated remediation via Config Rules → SSM Automation

---

Q9. How do you ensure audit trail integrity for regulatory compliance in banking?

Answer:

Banking regulators (RBI, SOX, PCI-DSS) require **immutable, complete, and tamper-proof audit trails**.

Implementation:

1. **CloudTrail:**
    - Organization trail — captures ALL API calls across ALL accounts
    - Management events + Data events (S3, Lambda, DynamoDB)
    - Log file validation enabled (SHA-256 digest chain)
    - Delivered to centralized Log Archive account
2. **Immutable Storage:**

```
resource "aws_s3_bucket_object_lock_configuration" "audit_logs" {
  bucket = aws_s3_bucket.audit_logs.id
  rule {
    default_retention {
      mode = "COMPLIANCE"  # Even root cannot delete
      years = 7            # Regulatory retention period
    }
  }
}
```

3. **Log Integrity:**
    - S3 versioning enabled (protects against overwrites)
    - S3 Object Lock in COMPLIANCE mode (protects against deletion)
    - CloudTrail log file validation (detects tampering)
    - Cross-account log delivery (separation of duties)
4. **Real-time Alerting:**
    - CloudWatch Metric Filters for critical events:
        - Root account login
        - IAM policy changes
        - Security group modifications

        - KMS key deletion scheduled
        - S3 bucket policy changes
    - SNS → PagerDuty/Slack for immediate response

5. **Forensic Readiness:**

    - Athena tables over CloudTrail S3 data for ad-hoc investigation
    - QuickSight dashboards for audit committee reporting
    - Automated compliance reports via Lambda + Config snapshots

---

Q10. How would you design a disaster recovery strategy for a bank with tiered application criticality?

Answer:

Not all banking applications need the same DR tier. I'd implement **tiered DR**:

Tier	Applications	RPO	RTO	AWS Strategy	Monthly Cost
Tier 1 (Critical)	Core banking, payments, ATM switch	0	<15 min	Multi-Region Active-Active (Aurora Global, DynamoDB Global Tables)	$$$$$
Tier 2 (Important)	Internet banking, mobile banking	<1 hr	<1 hr	Warm Standby (scaled-down infra in DR region)	$$$
Tier 3 (Standard)	CRM, HR, reporting	<4 hr	<4 hr	Pilot Light (DB replication + AMIs, no running compute)	$$
Tier 4 (Non-critical)	Dev/test, internal tools	<24 hr	<24 hr	Backup & Restore (S3 cross-region, snapshots)	$

Automated DR Testing:

- Monthly automated failover drills for Tier 1 (using Route 53 health check simulation)
- Quarterly full DR exercise for Tier 1-2
- Annual full DR exercise for all tiers
- GameDay exercises with chaos engineering (AWS FIS)
